Muhammad Noman Kamran

Final Project Report

Quick Submit

Quick Submit

National University of Computer and Emerging Sciences, Islamabad

Document Details

Submission ID
trn:oid:::1:3707087431

Submission Date
Nov 10, 2026, 10:10 AM GMT+5

Download Date
Nov 10, 2026, 10:12 AM GMT+5

File Name
Gin_rummy_Ai_final_report.docx

File Size
487.6 KB

12 Pages

3,228 Words

15,903 Characters

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (it may misidentify writing that is likely human generated as AI generated and likely AI generated as human generated) so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Final Project Report — Programming for AI

University Name: National University of Computer & Emerging Sciences – Karachi Campus

Course: Programming for AI

Section: BAI-3C

Project Title: Gin Rummy AI: A Pygame Card Game with Random Forest Decision Models

Submitted By: Muhammad Noman Kamran / 25k-0110

Muhammad Kashif Khan / 25k-0018

Manaam Siddiqui / 25k-0135

Submitted To: RAMSHA JUTT

Abstract

Gin Rummy AI is a card game developed in Python in which a single player will play against the computer. The computer has three key decisions to make: whether to draw from the deck or the discard pile, which card to discard, and when to knock. The models used in these selections are Random Forest models. Forests and the ID3 trees within the forests are created from scratch with NumPy and trained on CSV files generated by simulating Gin Rummy hands. The AI beat a random opponent 99.9% of the time and a greedy deadwood minimising opponent 52.6% of the time, in 1,000 games played against each of them.

1. Introduction

Gin Rummy is a game that's played by two players. You are trying to make sets (cards with the same number) or runs (cards in a row from the same suit). Any cards not part of a group is referred to as deadwood and their total value should be kept as low as possible.

While playing you have three important choices to make: whether to pick a card from the discard pile or the deck, which card to throw away, and when to knock or keep playing. Beginners simply guess and consequently lose.

This project will involve developing a fully functional, GUI based, Gin Rummy game with an AI opponent. The AI utilizes machine learning models to produce better choices. We did not have an appropriate dataset for our adaptation of the game so we made our own training data based on the game rules. Lastly, we compare our AI with basic computer players and see how it performs.

2. Objectives

1. Take the previous console implementation of Gin Rummy (cards, deck, hands, discard pile, knock logic) and port it to a Pygame game that uses card images and mouse movements for playing.
2. Create 3 training sets using simulated hands with the same rules that are followed during the game.
3. Train a Random Forest which determines whether a card should be drawn from the deck or the discard-pile.

4. Train a Random Forest that scores which card to discard, and another that decides between knocking, continuing and gin.
5. Replace the random opponent in the old code with the trained AI.
6. Compare the accuracy and win rate of the AI against a random player and a rule-based player.

3. System Design

System Overview

The project is split up into 4 sections. The game engine contains the cards, deck, hands, discard pile and the calculations for meld and deadwood. The table is drawn by the Pygame interface and the mouse is controlled by it. The dataset generator simulates hands and records the decisions in CSV files. The AI module trains the Random Forests on those files and is called by the game whenever the computer has to move.

The program's flow is:

Start → Main Menu → Deal 10 cards each and open one discard → Player turn (draw, then discard or knock) → AI turn (draw, discard, knock or gin) → Round ends → Scores updated → First to 100 points wins the match → Result screen → Main Menu.

Game Rules Used

- Players have 10 cards each from a 52-card deck. Cards are written with a suit letter followed by a number (e.g., D5 or H13).
- A set is 3 cards or more that have the same number. A run is a combination of 3 or more cards of the same suit in a row.
- The unmelded cards are added together to form Deadwood. The Ace is worth 1, and the Jack, Queen and King are each worth 10.
- A player can knock when the number of deadwood is 10 or less. Gin is the deadwood of 0.
- A normal knock scores the difference in deadwood. Gin scores the opponent's deadwood plus 25. If the opponent has equal or lower deadwood, it is an undercut and the opponent scores the difference plus 25.
- First player to get 100 points wins the match.

Algorithm

1. Run the program and display the main menu.
2. Deal 10 cards to the player and the AI, and one card to the discard pile.
3. Player turn: drag a card from the deck or discard pile to the hand, then drag a card to the discard pile or press Knock if permitted.
4. AI turn: the draw logic determines where to draw a card from, the discard logic determines which card to discard, and then the knock logic determines if it is gin or knock.
5. At the end of each round, update the scores. When a player reaches 100 points, show the final score; if not, let the player start a new round.
6. Go back to the main menu.

Input & Output

Input: mouse only. The player drags cards between the deck, the discard pile and the hand and clicks buttons for Knock, Sort, Theme, Music, Surrender and Next Round.

Output: Both hands and the table are shown, the number of cards left in the deck, the discard pile, the score, the player's current deadwood, status messages for each event, and a victory or defeat screen at the end of the game.

Project Structure

File / Folder	Purpose
main.py	Starts the game
gui.py	Pygame interface: menu, rules screen, table, drag and drop, animations, sound
gin rummy/cards.py	Card and Deck classes
gin rummy/play.py	Sorter (melds and deadwood), Hands, DiscardPile, game controller and the AI engine
model/IB3.py	ID3 decision tree written with NumPy
model/RandomForest.py	Random Forest built on the ID3 class
gin_rummy_dataset_generator.py	Creates the three CSV datasets (seed 42)
evaluate_models.py	Accuracy tests, charts and win-rate simulation
test_game.py	Unit tests for the game logic and AI
dataset/	The three generated CSV files
evaluation_results/	Charts, summary text and report produced by the evaluation

4. Implementation

Language: Python 3

Libraries: pygame-ce (graphics, mouse and sound), NumPy (trees and forests), Pandas (reading the CSV files), Matplotlib (charts). Pickle is used to save trained models.

Tools: VS Code, Git and GitHub.

Key Features

- Main menu with Start Game, card theme, rules screen and a music switch.
- Four card themes that are loaded from the CuteCards sprite sheets: Classic Royale, Cute Pixel, Outlined Pixel and Traditional.
- Controls for drag and drop drawing from the deck or discard pile and for discarding.
- AUTO (melds first), RUNS, SETS and RANK are the four sorting modes for the hand.
- Knock / Gin button which only appears when the player can knock.
- Card move animations, screen shake on knock/gins, sound effects and looping background music.
- Surrender button, and a Next Round button after each round.

- When the deck is empty it is refilled from the discard pile.

Datasets

Since there was no readily available dataset for our simplified rules, it was generated using gin_rummy_dataset_generator.py. It plays a few turns of greedy play in order to get to a realistic mid-game hand then looks ahead using Monte Carlo sampling of the cards that it doesn't see. The result of the look-ahead becomes the label. A fixed seed of 42 allows the files to be recreated exactly.

Dataset	Rows	Features	Label
draw_decision_dataset.csv	10,000	13	take_discard: 0 = deck, 1 = discard pile (look-ahead of 25 sampled deck cards)
discard_decision_dataset.csv	24,000	15	best_discard: 1 for the card with the lowest expected deadwood (15 sampled draws)
knock_gin_decision_dataset.csv	3,000	10	decision: 0 = keep playing, 1 = knock, 2 = gin (up to 20 sampled opponent hands)

The draw data is not evenly distributed, with approximately 83% of the data rows labeled deck and 17% labeled discard. The knock file also includes two columns for win probability (these are removed before training). The features are what a human player looks at: how low the deadwood is, how close the hand is to a meld and how useful the card in question is.

Draw dataset (13 features, label: take_discard)

Feature	Meaning	Why it is used
turn_number	Round progress	Early and late turns need different risk
deck_size	Cards left in the deck	Shows how much time is left
deadwood_now	Current deadwood	Shows how much the hand must improve
cards_in_sets, cards_in_sequences	Cards already melded	Shows how complete the hand is
cards_unmelded	Loose cards	These add deadwood
num_pairs	Pairs of the same number	One card away from a set
num_suit_connectors	Same-suit cards 1–2 apart	One card away from a run
discard_top_number	Number on the top discard	Sets the card's value and which melds it fits
top_same_number_in_hand	Cards in hand with its number	Taking it may complete a set
top_suit_adjacent1, top_suit_adjacent2	Same-suit cards 1–2 away from it	Taking it may extend a run
deadwood_gain_if_take	Deadwood saved by taking it	The clearest sign that taking it helps

Discard dataset (15 features, label: best_discard)

Feature	Meaning	Why it is used
turn_number	Round progress	Same reason as above
deadwood_before_discard, deadwood_after_discard	Deadwood before and after the throw	Shows what throwing the card does
deadwood_reduction	Difference of the two	Measures the benefit directly
card_number	Number on the card	High cards cost up to 10 points
card_in_set, card_in_sequence	Card is inside a meld	Throwing it breaks the meld
same_number_in_hand	Other cards with the same number	May become a set
suit_adjacent1, suit_adjacent2	Same-suit cards 1–2 away	May become a run
pairs_left, suit_connectors_left	Pairs and near-runs left after the throw	Shows remaining meld potential
outs_after_discard	Estimate of cards that could improve the hand	Shows how likely the hand is to improve
outs_vs_best_option, deadwood_vs_best_option	Comparison with the best card in the same hand	Lets the model compare cards within one hand

Knock/gin dataset (10 features, label: decision)

Feature	Meaning	Why it is used
my_deadwood	Current deadwood	A knock needs 10 or less, and lower is safer
turn_number, deck_size	Round progress, cards left	Shows how long the game can still go
cards_in_sets, cards_in_sequences, cards_unmelded	Hand structure	Shows how stable the hand is
highest_deadwood_card	Highest loose card (max 10)	One big loose card makes the hand risky
num_pairs, num_suit_connectors	Meld potential	Shows how likely the hand is to improve
gin_chance_next_draw	Estimated chance of reaching gin	If high, waiting may beat knocking

Random Forest Model

The tree and the forest are both our own code. For each ID3 tree, the splits are chosen based on entropy and information gain. The entropy is a measure of the amount of mixing in a set of samples, and p(c) is the proportion of samples in class c:

$$H(Y) = -\Sigma\, p(c) \cdot \log_2 p(c)$$

If a group is divided into two parts, left and right, then the conditional entropy is the average entropy of the two parts, according to the size of the parts. The information gain is how much the entropy drops after the split:

$$\text{Conditional entropy} = \Sigma \left(|Y_i| / |Y| \right) \cdot H(Y_i)$$

$$\text{Information gain (IG)} = H(Y) - \text{Conditional entropy}$$

The entropy in the code is given by -np.sum(prob*np.log2(prob)). The info_gain function is the sum of the entropies of each part scaled by len(y) / len(prev_y) then subtracted from the entropy before the split. The split with the highest gain is chosen.

There are 10 trees in the forest. For each tree, 70% of the features are used randomly and a bootstrap sample of the rows is taken with a maximum depth of 8 and minimum of 4 samples to split. The class most trees vote for becomes the final answer.

In our project, 10 samples (7 deck, 3 discard) give H = 0.8813 bits. A split on dw_gain > 4 results in a left group of 6 samples all deck (H = 0) and a right group of 4 samples (1 deck, 3 discard) that have an H of 0.8113. The weighted entropy after the split is 0.3245, so IG = 0.8813 − 0.3245 = 0.5568 bits.

How the AI Plays

The forests are combined with some fixed rules, so the AI is a hybrid and not a pure model.

- **Draw.** If the AI can remove 8 or more deadwood, or remove deadwood and have 2 cards of the same value in the hand, or 2 cards that are neighbours in suit in the hand, it will take the discard immediately. Otherwise, the draw forest determines it and a “take” answer is only possible if it removes deadwood or fits a card that is already in hand.
- **Discard.** All of the 11 cards in the hand are given a score and the highest one is thrown. A card in a meld loses 5000 points so it is almost never thrown. The score goes up by 25 × (100 − deadwood left after the throw) and by 4 × the card value (capped at 10). It decreases by 12 for each card with the same number and by 8 for each same-suit neighbour. Lastly, if the forest marks the card as the best discard, it adds 40.
- **Knock.** The AI never knocks above deadwood 10. At deadwood 0 it declares gin, and at 4 or less it knocks automatically. Between 5 and 10, the knock forest decides.

The forests are trained from the csv files by the engine if there is no saved model file; the first 2,000 rows of each csv file are used, with a train-test split of 80/20; the trained forests are saved in model/trained_models.pkl.

Code Snippet

Entropy and information gain from model/IB3.py:

```
def entropy(self,y):
    if len(y) <=1:
        return 0
    totalcount = np.bincount(y)
    prob = totalcount[np.nonzero(totalcount)]/len(y)
    if len(prob)<=1:
        return 0
    return -np.sum(prob*np.log2(prob))

def info_gain(self,prev_y , current_y):
    cond_entopy =0
    for y in current_y:
        cond_entopy += self.entropy(y)*len(y)/len(prev_y)
    info_gain = self.entropy(prev_y)-cond_entopy
    return info_gain
```

Knock decision from AIRandomForestEngine in gin rummy/play.py (shortened):

```
def predict_knock(self, real_deadwood, hand_cards, turn_num, deck_size):
    if real_deadwood > 10:
        return False
    if real_deadwood == 0:
        return True  # Always trigger GIN immediately
    if real_deadwood <= 4:
        return True  # Strong knock, very difficult to undercut
    ...
    pred = self.knock_model.predict(features)[0]
    return bool(pred in [1, 2])
```

Sample Output

Screens captured from the running game:



Figure 1: Main menu



Figure 2: Game table at the start of a round (10 cards each, 31 cards left in the deck)

5. Testing & Results

Unit Tests

test_game.py contains 19 unit tests. We ran the file for this report and all 19 passed.

Group	Tests	What is checked
Deck basics	3	52 unique cards, shuffling, reset
Deck recycling	2	Discards go back into an empty deck; the AI hand stays at 10 cards when the deck is empty
Melds and deadwood	5	Pure sets, pure runs, best meld choice, ace and face-card values, a 10-card gin hand
Knocking and scoring	3	Gin bonus of 25, undercut bonus of 25, illegal knock is rejected
AI behaviour	2	The AI never discards a melded card; it takes a discard that extends a run
Hand sorting	4	Rank, runs and sets sorting; the sort mode cycles correctly

Model Accuracy

evaluate_models.py trains each forest, using the first 80% of its csv file, and tests it using the last 20%. Results are below.

Draw model

Model	Accuracy	Precision	Recall	F1
Random Forest	97.5%	95.3%	88.7%	91.9%
Always take from deck	84.0%	—	—	—
Take if deadwood drops	85.0%	—	—	—

Discard model

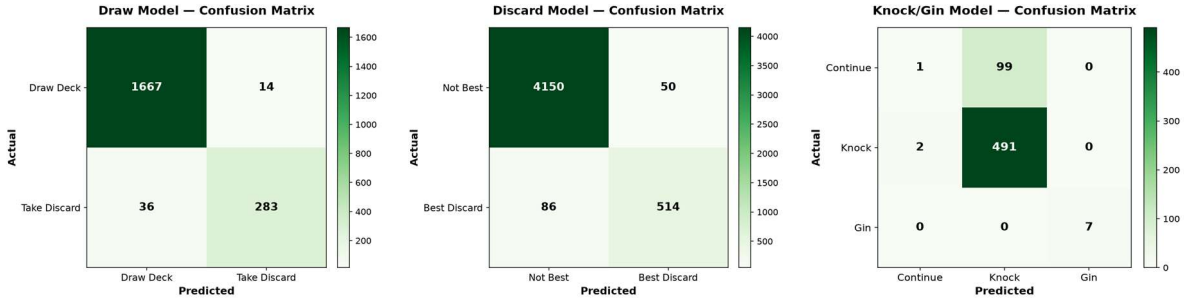
Model	Accuracy
Random Forest	97.2%
Highest card rule	86.6%
Random guess	49.7%
Lowest deadwood rule	39.8%

Accuracy for the discard model is counted per card, and only about one card in eight is labeled as the best discard, so a high accuracy is easier to reach than it looks. The baselines here are simple rules on single columns of the test data, so they are not the same as the greedy player used in the game simulation below.

Knock / gin model

Model	Accuracy	Balanced accuracy	Macro F1
Random Forest	83.2%	66.9%	64.2%
Always knock	83.3%	66.7%	63.6%

This is the hardest of the three decisions. In about 82% of the rows, Knock is the correct label and the forest ends up nearly level with the “always knock” baseline.



Draw model *Discard model* *Knock/gin model*

Figure 3: Confusion matrices

Feature Importance

In the draw model, deadwood_gain_if_take is the most important factor, then discard_top_number and top_same_number_in_hand. Deadwood_after_discard and deadwood_reduction are the most important features for the discard model, while card_number is also significant due to the face cards being valued at 10. My_deadwood, gin_chance_next_draw, and deck_size are the top three features for the knock model.

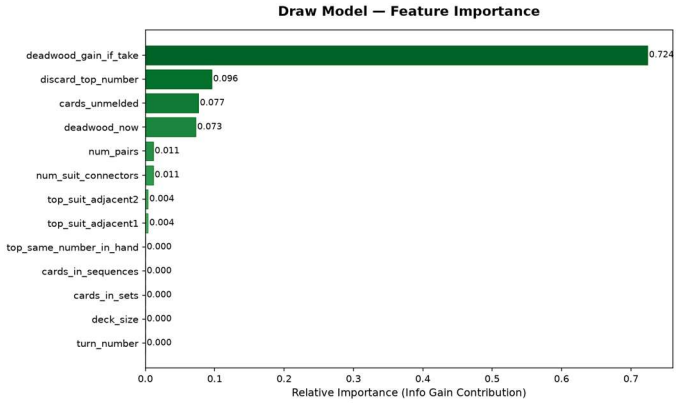


Figure 4: Draw model feature importance

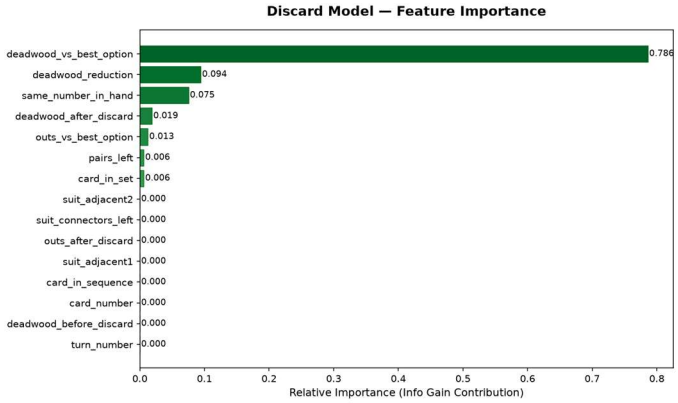


Figure 5: Discard model feature importance

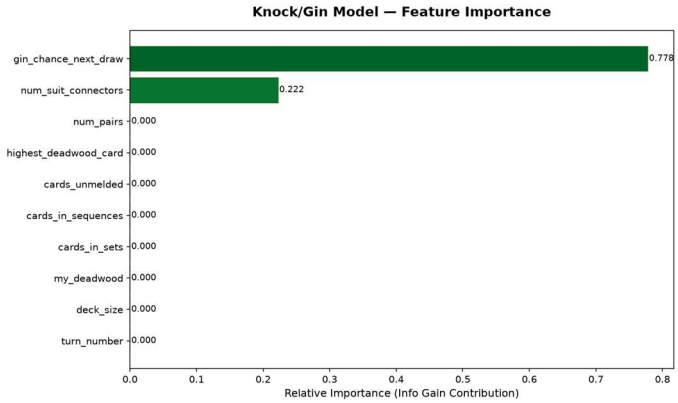


Figure 6: Knock/gin model feature importance

Win-Rate Simulation

The AI played 1,000 full games against each baseline. The random player draws and discards at random and never knocks. The greedy player always minimises deadwood and knocks whenever it can.

Metric	AI vs Random	AI vs Greedy
AI wins / opponent wins / draws	999 / 1 / 0	526 / 474 / 0
AI win rate	99.9%	52.6%
Average AI deadwood	6.7	14.5
Average opponent deadwood	58.5	16.7
Average turns per game	8.1	6.5

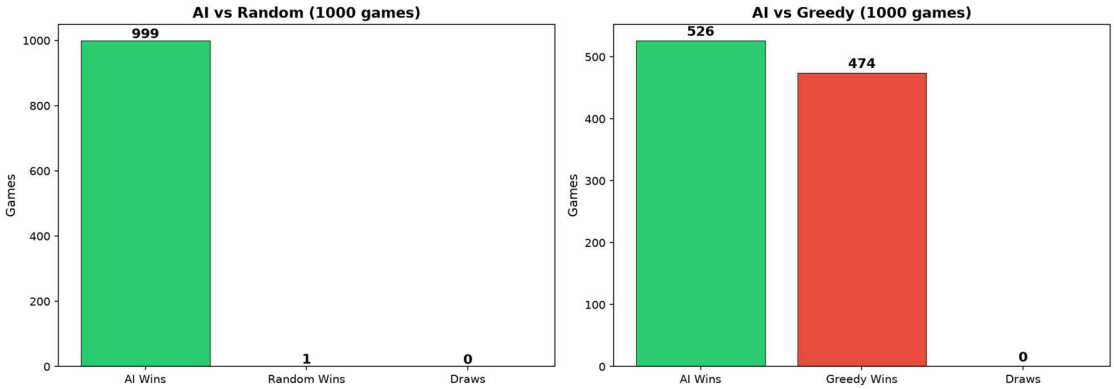


Figure 7: Win rate of the AI against both baselines

The AI is far ahead of a random player. The edge against the greedy player is minimal and it is better to say it is about equal with a little advantage, rather than to say it as a clear win.

6. Conclusion, Limitations & References

Conclusion

The project provides a playable Gin Rummy game with card graphics, drag and drop controls, scoring and an AI opponent. The code for the Random Forest and ID3 algorithms was written in-house, the training data was generated from the rules of the game, and two baselines were used to test them. The draw and discard models are able to learn their tasks effectively on the test data. The knock model gives almost no benefit over the "always knock" rule and in full games, the AI only slightly beats the greedy player.

Limitations

- The rules are simpler: no layoffs, card values follow the code.
- Labels are obtained from simulated look-ahead, NOT from expert human play.
- Each decision is taken individually. The AI makes its decisions one turn at a time, and doesn't track which cards the other player has picked up.
- The knock model is only slightly better than the "always knock" model.
- The AI also follows some rules and has a scoring formula, so the play is not entirely based on the models.
- When training on its own, the game only utilizes 2,000 rows of each dataset and no systematic search of hyperparameters was performed.
- The AI is tested on bots only. It is not measured for its strength against the skilled human players.
- The datasets were created using 7-card hands, and the actual game is played with 10-card hands.
- Two discard features have been calculated based on the best possible card and the actual discard accuracy in-game may be slightly lower.
- No online multiplayer is available.

Future Enhancements

- Search for better tree count and depth settings.
- Train the in-game models with the full datasets.
- Add opponent tracking so that the AI can use the cards the other player has taken.
- Test the AI against human players.

References

- Pygame documentation: https://www.pygame.org/docs/
- NumPy documentation: https://numpy.org/doc/
- Pandas documentation: https://pandas.pydata.org/docs/
- Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32.
- Gin rummy rules (Wikipedia): https://en.wikipedia.org/wiki/Gin_rummy
- CuteCards asset pack by Dani Maccari (card images): https://dani-maccari.itch.io/